Classical Cryptanalysis Toolkit

A comparison of functional and constraint-based paradigms

Task 1 · Cryptanalysis Toolkit in Haskell

Task 2 · Cryptarithm Solver in Python with Z3

Elisha Benjamin Sreepalle

github.com/ebenjamin12/classical-cryptanalysis-toolkit

October 2026

Contents

1 Overview 3

1.1 Repository structure 3

1.2 Summary of results 3

2 Task 1 — Cryptanalysis Toolkit (Haskell) 4

2.1 Why Haskell 4

2.2 Caesar cipher 5

2.3 Vigenere cipher 5

2.4 Frequency analysis 6

2.5 Kasiski examination 7

2.6 Testing 7

3 Task 2 — Cryptarithm Solver (Python + Z3) 9

3.1 Why a constraint solver 9

3.2 Puzzle generation 9

3.3 Testing 10

3.4 Performance: Z3 against brute force 11

4 Paradigm comparison 13

5 Notes on the implementation 13

5.1 Reproducing these results 13

6 References 14

1 Overview

This report documents two command-line programs written to explore how the choice of programming paradigm shapes a solution. Both solve problems from classical cryptography, but they approach them from opposite directions.

The first is a cryptanalysis toolkit written in Haskell. It implements the Caesar and Vigenere ciphers, and two attacks against them: chi-squared frequency analysis for recovering an unknown Caesar key, and Kasiski examination for estimating a Vigenere key length. The code is purely functional throughout.

The second is a cryptarithm solver written in Python using the Z3 SMT solver. It treats puzzles such as SEND + MORE = MONEY as constraint satisfaction problems, stating what a valid solution must look like rather than how to search for one. A brute-force enumerator is retained as a fallback and as an independent check on the solver's answers.

Every figure in this report is a capture of the programs actually running, and every timing was measured rather than estimated. Where results contradict expectations, this is noted rather than smoothed over.

1.1 Repository structure

```
classical-cryptanalysis-toolkit/
├── README.md
├── LICENSE
├── task1-haskell-cryptography/
│   ├── SimpleCryptography.hs        # ciphers and cryptanalysis
│   ├── cryptanalysis-toolkit.cabal  # build definition (base only)
│   └── img/
├── task2-cryptarithm-z3/
│   ├── CryptarithmSolver.py         # Z3 encoding + fallback
│   ├── benchmark.py                 # engine comparison
│   ├── requirements.txt
│   └── img/
└── docs/
```

1.2 Summary of results

	Task 1 — Haskell	Task 2 — Python + Z3
Paradigm	Functional	Logic / constraint
Modes implemented	5	3
Automated tests	25 passing	9 passing
Dependencies	none (base only)	z3-solver
Lines of code	≈290	≈250

2 Task 1 — Cryptanalysis Toolkit (Haskell)

An interactive toolkit implementing two classical ciphers and two attacks against them. The program presents a menu and loops until the user exits.

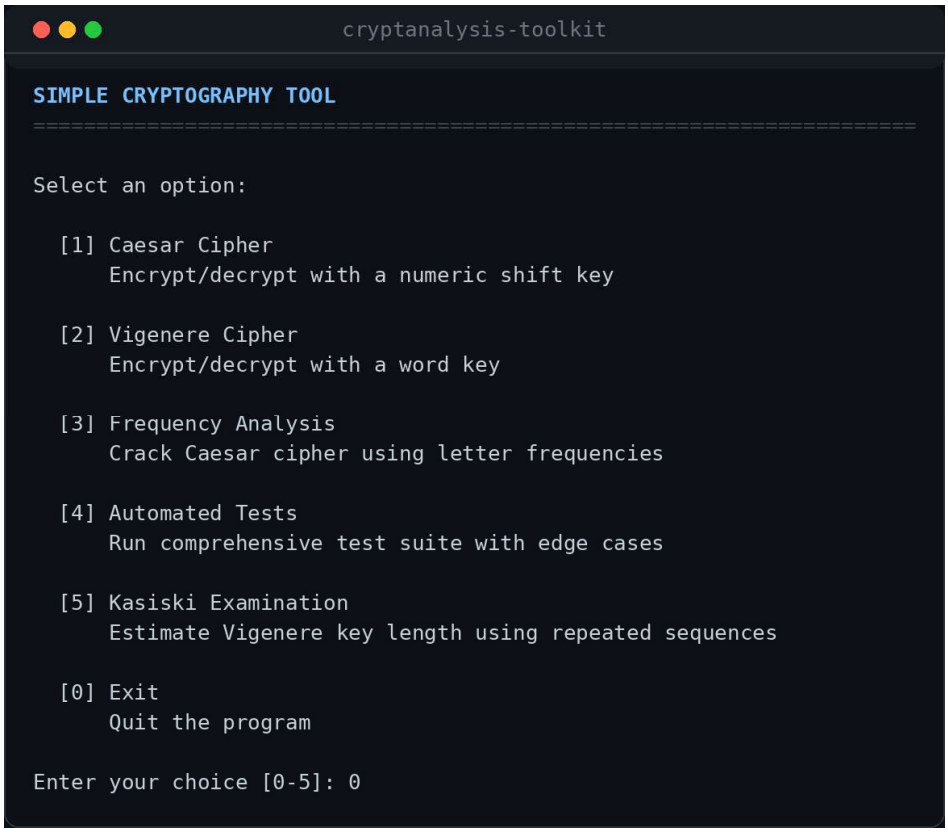


Figure 1 — Main menu, showing all five modes.

2.1 Why Haskell

Cryptographic primitives are mathematical transformations, which maps onto a functional language more directly than an imperative one. Four properties of the language were decisive:

- **Referential transparency.** `caesarEncrypt 3 "HELLO"` always yields `"KHOOR"` with no side effects, which eliminates a whole class of state-related bugs that imperative cipher implementations are prone to.
- **List processing.** Cryptanalysis is largely iterating, filtering and counting characters. A comprehension such as `[(c, length (filter (== c) lowered)) | c <- ['a'..'z']]` states the intent directly instead of describing a loop that produces it.
- **Type safety.** Signatures like `shiftChar :: Int -> Char -> Char` are contracts checked at compile time, catching category errors that a dynamically typed language would surface only at runtime.
- **Lazy evaluation.** The key stream `keyShifts key = cycle alphaKey` is conceptually infinite. Laziness means only the elements actually consumed during encryption are computed, so the list is never materialised in memory.

2.2 Caesar cipher

Each letter is shifted by a fixed amount using modular arithmetic. Decryption reuses encryption with a negated key, exploiting the group structure: applying `k` and then `−k` returns the identity, so no separate decrypt routine is needed.

```
shiftChar :: Int -> Char -> Char
shiftChar n c
  | isUpper c = chr (((ord c - ord 'A' + n) `mod` 26) + ord 'A')
  | isLower c = chr (((ord c - ord 'a' + n) `mod` 26) + ord 'a')
  | otherwise = c

caesarDecrypt key = caesarEncrypt (negate key)
```

Case is preserved and non-alphabetic characters pass through untouched, so punctuation and spacing survive a round trip.

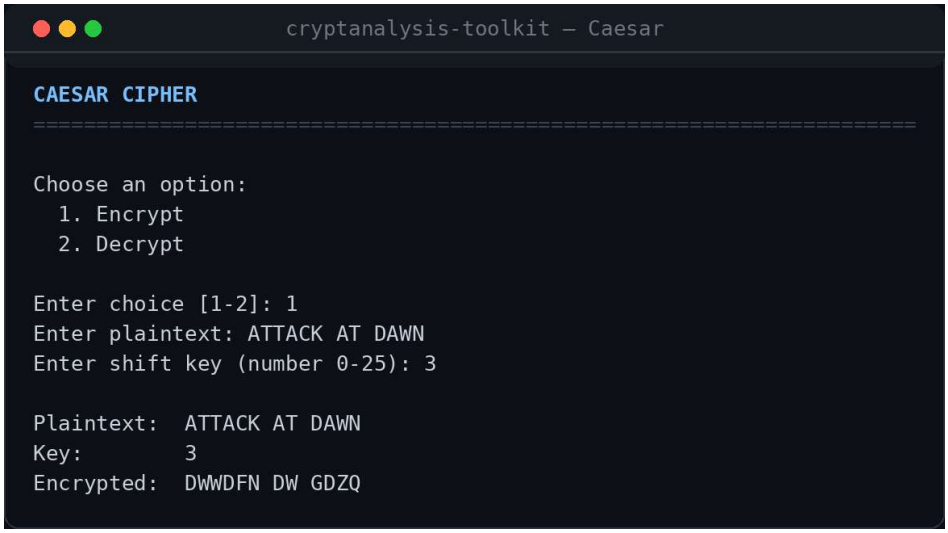


Figure 2 — Caesar encryption of ATTACK AT DAWN with a shift of 3.

2.3 Vigenere cipher

The key is converted into a cycling list of shifts — `LEMON` becomes `[11, 4, 12, 14, 13]`. Only alphabetic characters consume a key position, so spaces and punctuation do not knock the key out of alignment with the plaintext.

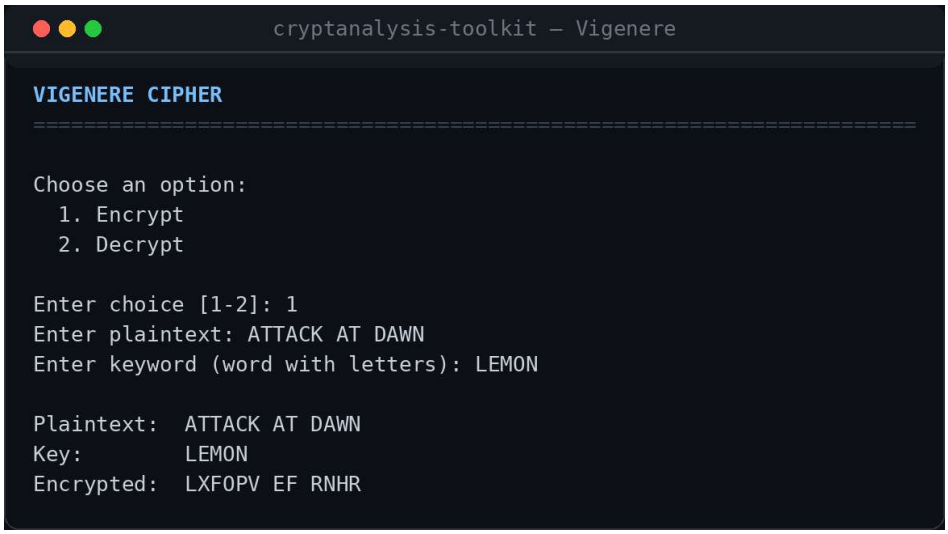


Figure 3 — Vigenere encryption reproducing the textbook result LXFOPV EF RNHR.

2.4 Frequency analysis

Given a Caesar ciphertext with an unknown key, each of the 26 possible shifts is applied in turn and the resulting letter distribution is scored against English baseline frequencies using a chi-squared statistic:

$$\chi^2 = \Sigma\ (observed - expected)^2\ /\ expected$$

A low score means the decrypted distribution closely matches English, so the lowest-scoring shift is the best candidate. The run below recovers the plaintext exactly.

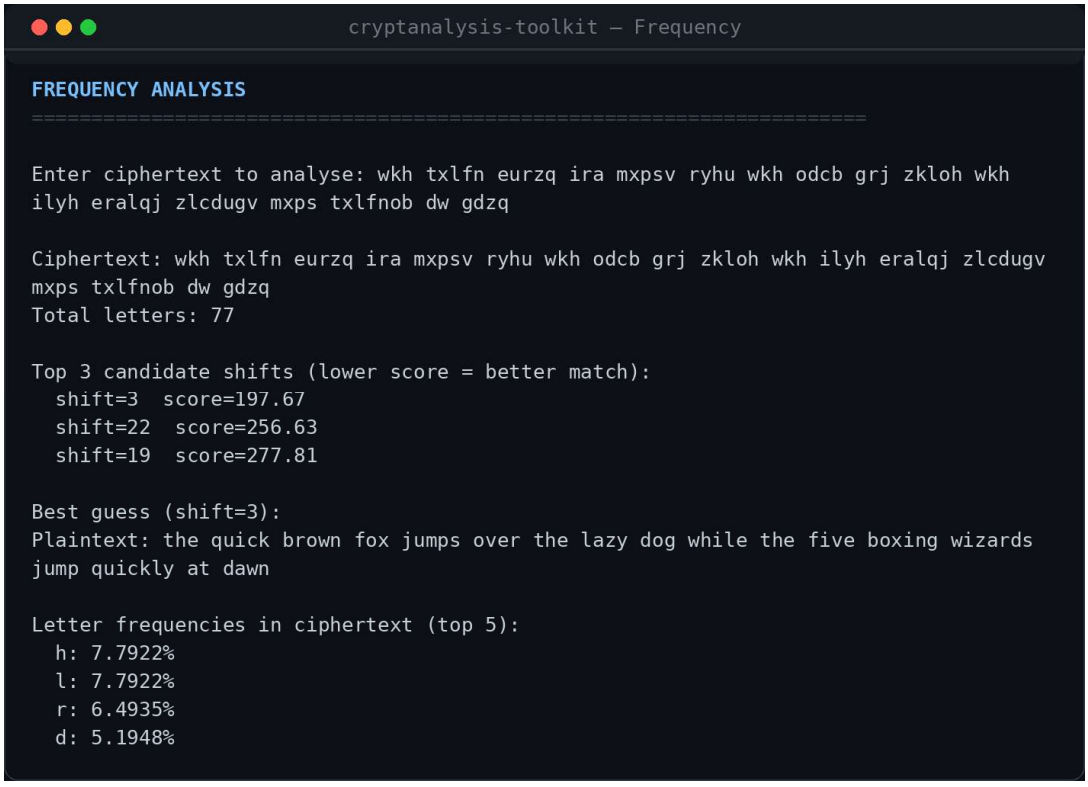


Figure 4 — The correct shift of 3 scores 197.67 against 256.63 for the nearest rival.

2.5 Kasiski examination

This is the extension beyond the basic requirement. Repeated trigrams in a Vigenere ciphertext tend to arise where the same plaintext meets the same part of the key, so the distances between those repeats are usually multiples of the key length. The tool collects the distances, computes their divisors, and ranks candidate key lengths by how many distances each one explains.

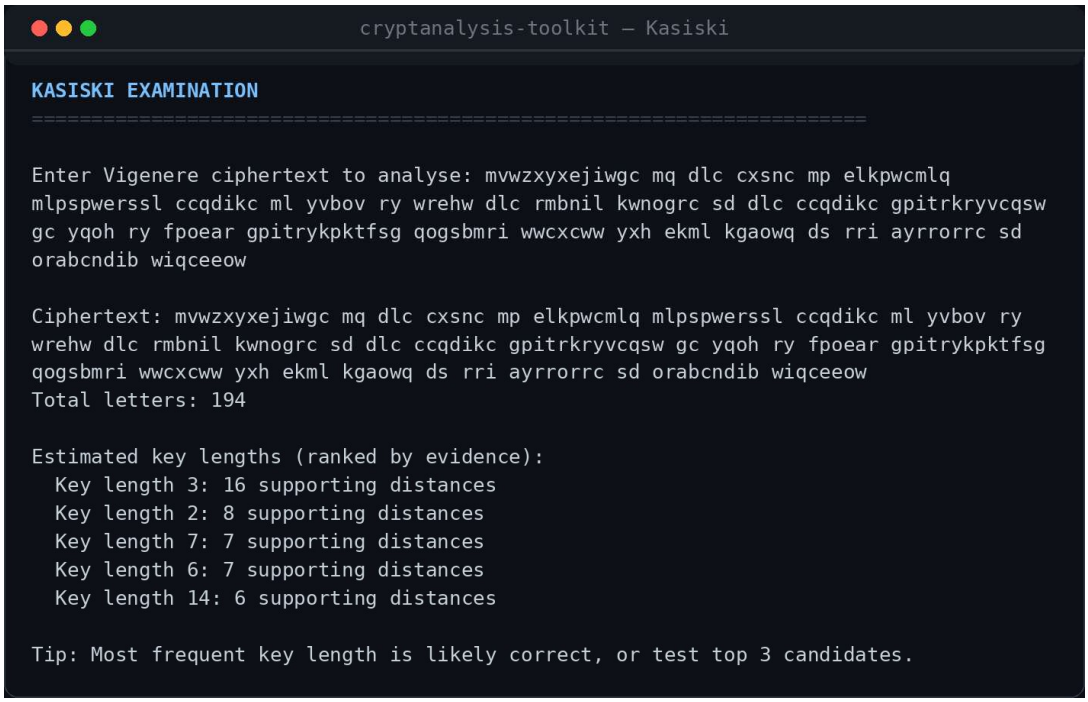


Figure 5 — Against 194 letters enciphered with a three-letter key, length 3 is identified with twice the evidence of any rival.

A limitation worth stating

Kasiski narrows the field; it does not prove an answer. On a highly periodic plaintext every divisor of the repeat interval explains every distance equally well, so the ranking ties across 2, 3, 4, 6 and 8 with identical scores. Natural language breaks those ties because its trigram repeats fall at irregular intervals, which is why the natural-English example above produces a clear winner.

The correct reading of the output is therefore "test the top few candidates", which is why the tool prints five rather than asserting one.

2.6 Testing

The suite covers five groups: Caesar (7 cases), Vigenere (6), frequency analysis (3), edge cases (5) and Kasiski (4). The edge cases are the most valuable — empty strings, single characters, negative shifts, pure punctuation, and alphabet wraparound.



Figure 6 — All 25 tests passing.

The high pass rate is partly a property of the language. Immutability and static typing catch a large share of mistakes at compile time, so the runtime tests are free to concentrate on boundary conditions and on mathematical invariants such as modular closure and the inverse-function property.

3 Task 2 — Cryptarithm Solver (Python + Z3)

A cryptarithm assigns a distinct digit to each letter such that an arithmetic statement holds — the classic being `SEND + MORE = MONEY`. Leading digits may not be zero, and no two letters may share a digit.

3.1 Why a constraint solver

A cryptarithm is a constraint satisfaction problem, and encoding it as one keeps the puzzle's semantics separate from the search strategy. The program states what must be true and leaves the solver to find an assignment:

Constraint	Z3 encoding
Each letter is a digit	v >= 0, v <= 9
All letters differ	z3.Distinct(...)
No leading zeros	vs[c] != 0 for each leading letter
The arithmetic holds	val(w1) + val(w2) == val(w3)

Z3 then applies DPLL with conflict-driven clause learning, abandoning a branch as soon as a partial assignment becomes inconsistent rather than enumerating permutations blindly.

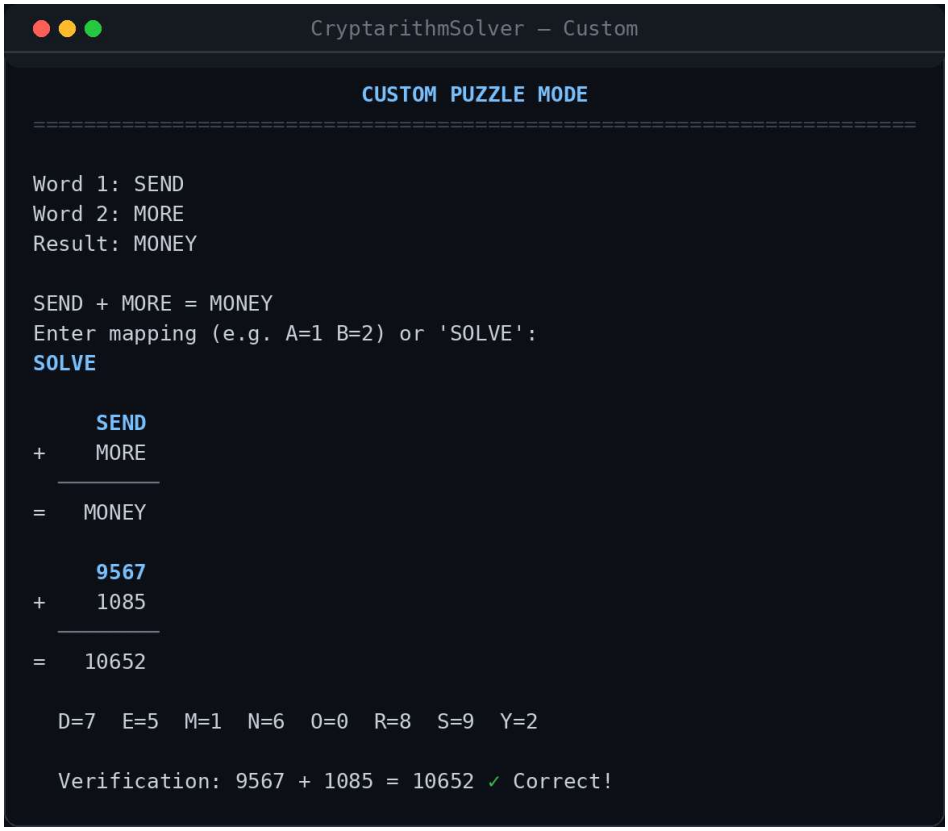


Figure 7 — Solving SEND + MORE = MONEY, giving the unique solution 9567 + 1085 = 10652.

3.2 Puzzle generation

Two words of three to five letters are drawn from a word list, digits are assigned by shuffling, leading zeros are repaired by swapping, and the sum is mapped back into letters. A candidate is kept only if the result is itself a real word with a consistent letter-to-digit mapping, which is why most attempts are discarded.

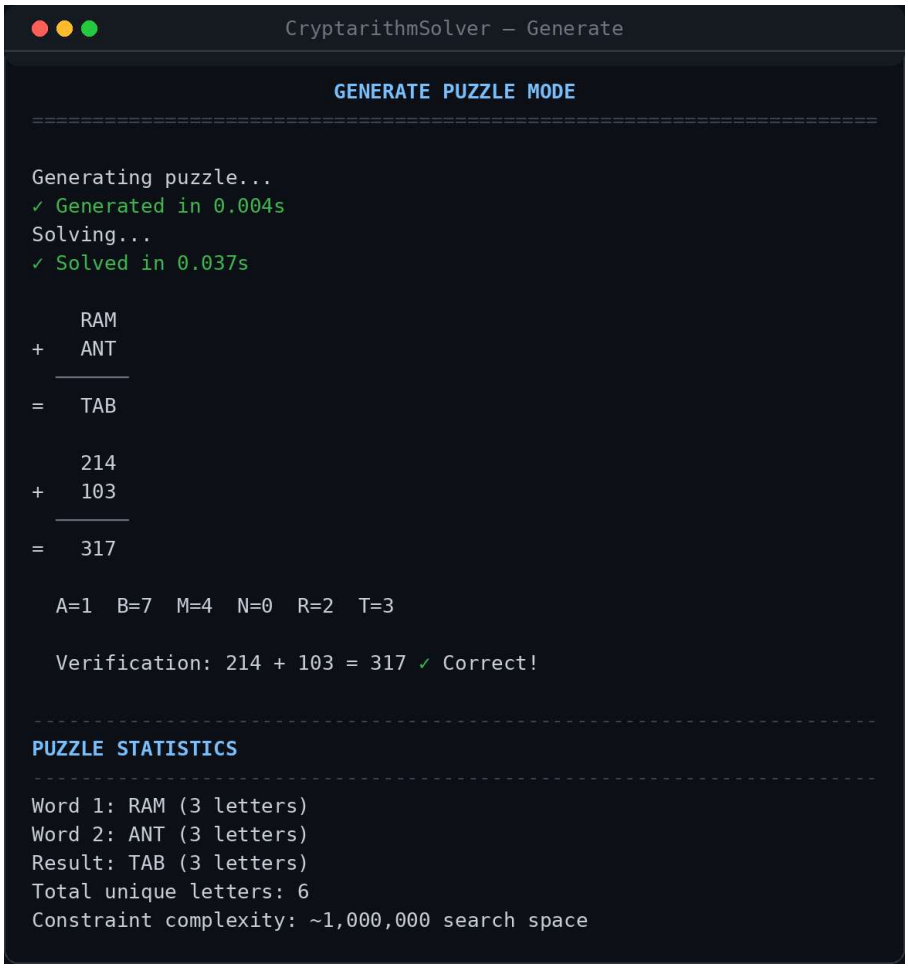


Figure 8 — A generated puzzle, solved and verified.

3.3 Testing

Nine cases cover both solvable and unsolvable puzzles. The unsolvable ones matter as much as the rest: a solver that never returns "no solution" is not trustworthy.

#	Puzzle	Letters	Outcome
1	SEND + MORE = MONEY	8	9567 + 1085 = 10652
2	TWO + TWO = FOUR	6	765 + 765 = 1530
3	CROSS + ROADS = DANGER	9	96233 + 62513 = 158746
4	A + B = C	3	solvable
5	AA + B = AC	3	solvable
6	ABCD + EFGH = IJKL	12	rejected — more than 10 letters

#	Puzzle	Letters	Outcome
7	ONE + ONE = MANY	6	703 + 703 = 1406
8	SIX + TEN = SIX	6	no solution — needs TEN = 0
9	ART + ART = CART	4	no solution

Case 6 never reaches the solver. With twelve distinct letters and only ten digits available, the pigeonhole principle rules it out and the function returns early — a cheap check that avoids an expensive search.



Figure 9 — All 9 tests passing.

3.4 Performance: Z3 against brute force

The accompanying `benchmark.py` runs both engines over the same nine puzzles, reports wall-clock time for each, and verifies that the two agree on whether a solution exists.

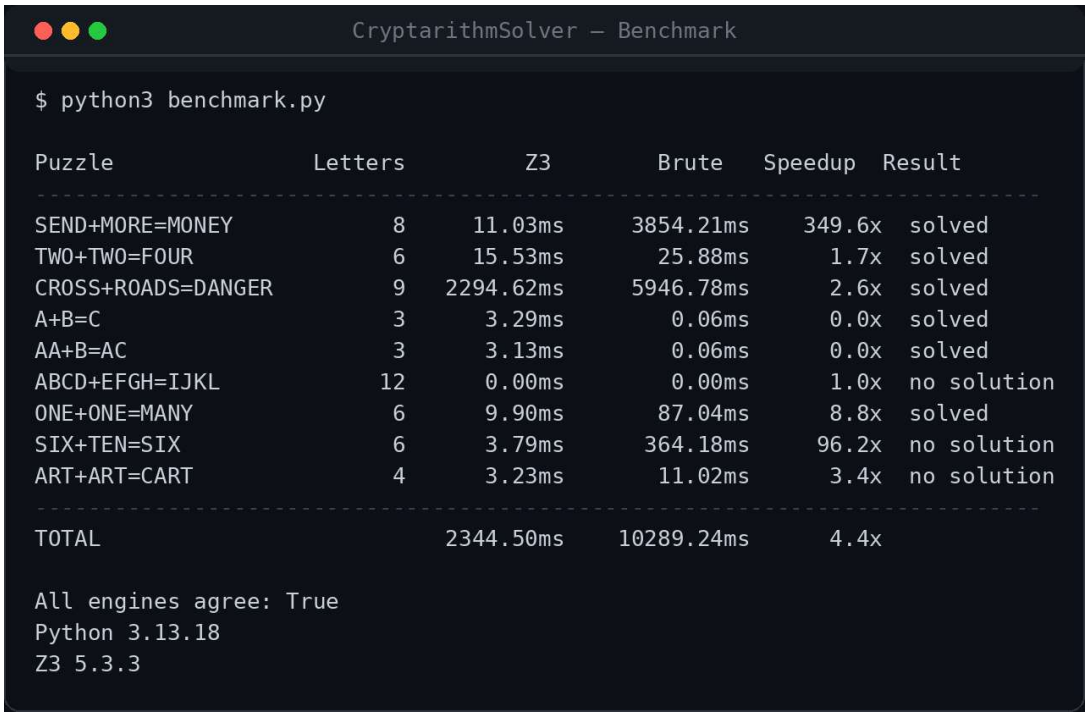


Figure 10 — Measured comparison of the two engines.

The headline finding is that the speedup depends entirely on the size of the search space, and that quoting a single figure would be misleading:

Puzzle	Letters	Z3	Brute force	Speedup
SEND + MORE = MONEY	8	11 ms	3,854 ms	**350×**
SIX + TEN = SIX	6	4 ms	364 ms	**96×**
ONE + ONE = MANY	6	10 ms	87 ms	9×
CROSS + ROADS = DANGER	9	2,295 ms	5,947 ms	2.6×
A + B = C	3	3.3 ms	0.06 ms	**0.02× slower**

Three observations follow from this:

1. Z3 wins where it matters. On eight distinct letters, brute force walks P(10,8) ≈ 1.8 million permutations; the solver prunes almost all of that away.
2. Z3 loses on trivial puzzles. With three letters there are only a few hundred permutations, and building the constraint system costs more than simply checking them all. Declarative encoding is not free.
3. CROSS + ROADS = DANGER is the awkward case. Nine letters, and Z3 still needs 2.3 seconds — a satisfiable instance with a sparse solution space forces considerable search even with clause learning.

Both engines agreed on all nine puzzles, which is the useful correctness signal: the fallback independently confirms the SMT encoding rather than merely duplicating it.

Measured on Python 3.13.18 with Z3 5.3.3; best of three runs for Z3, single run for brute force. Absolute timings vary by machine.

4 Paradigm comparison

	Functional (Haskell)	Constraint (Z3)
Problem shape	Stream transformation	Constraint satisfaction
You write	How to transform the data	What must be true
Correctness from	Compile-time types, pure functions	Solver-verified model
Cost profile	Predictable, linear in input length	Variable — cheap or exponential
Fails by	Not compiling	Taking too long

The contrast is sharpest in what each paradigm makes easy. The Haskell code never states what a cipher is, only how characters move. The Z3 code never states how to search, only what a valid assignment looks like.

Neither would serve the other's problem well. Expressing a cipher as constraints would be perverse, since the transformation is already a direct computation with nothing to search for. Expressing a cryptarithm as a stream transformation would mean writing the search by hand, which is exactly the work the solver exists to remove. The paradigm is not a stylistic preference; it follows from the shape of the problem.

5 Notes on the implementation

This code began as coursework and was later reorganised for publication. Three changes were made to the Haskell source after submission, none of which alter the algorithms:

- Added the missing type signature for `printTest`, so the module compiles cleanly under `-Wall`.
- Set `hSetEncoding stdout utf8` in `main`. The tick and cross glyphs used in the test output caused the program to abort on systems whose default locale is not UTF-8.
- Set `hSetBuffering stdout NoBuffering`, so prompts written with `putStr` appear before the program waits for input rather than after it.

The second of these was found only by running the program under a non-UTF-8 locale, which is a reminder that a test suite reporting 100% tells you about the logic, not about the environment it runs in.

5.1 Reproducing these results

```
# Task 1 — no external dependencies
cd task1-haskell-cryptography
cabal run

# Task 2
cd task2-cryptarithm-z3
pip install -r requirements.txt
python3 CryptarithmSolver.py
python3 benchmark.py
```

6 References

de Moura, L. & Bjørner, N. (2008). Z3: An Efficient SMT Solver. In Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2008), pp. 337–340. Springer.

Friedman, W. F. (1976). The Index of Coincidence and its Applications in Cryptography. Aegean Park Press.

Hutton, G. (2016). Programming in Haskell (2nd ed.). Cambridge University Press.

Kahn, D. (1967). The Codebreakers: The Story of Secret Writing. Macmillan.

Menezes, A., van Oorschot, P. & Vanstone, S. (1997). Handbook of Applied Cryptography. CRC Press.

Singh, S. (1999). The Code Book: The Secret History of Codes and Code-breaking. Fourth Estate.

Stallings, W. (2017). Cryptography and Network Security: Principles and Practice (7th ed.). Pearson.

Tsang, E. P. K. (1993). Foundations of Constraint Satisfaction. Academic Press.

Haskell 2010 Language Report. https://www.haskell.org/onlinereport/haskell2010/

Z3 Documentation and API Reference. https://github.com/Z3Prover/z3/wiki